

密级状态：绝密 () 秘密 () 内部 () 公开
(√)

智能视频分析（ROCKIVA）SDK 开发指南

（技术部，图形计算平台中心）

文件状态： [] 正在修改 [√] 正式发布	当前版本：	V2.0.0BETA
	作 者：	AI Team
	完成日期：	2024-10-14
	审 核：	熊伟
	完成日期：	2024-10-14

瑞芯微电子股份有限公司

Rockchips Semiconductor Co., Ltd

(版本所有, 翻版必究)

更新记录

版本	修改人	修改日期	修改说明	核定人
V2.0.0BETA	杨华聪	20241014	1. 新增 SDK 模型说明	熊伟

目 录

1	概述	4
2	SDK 模型说明	5
2.1	检测模型说明	5
3	SDK 开发说明	6
3.1	通用模块	7
3.1.1	功能描述	7
3.1.2	API 参考	9
3.2	检测模块	30
3.2.1	功能描述	30
3.2.2	API 参考	32
3.3	周界模块	35
3.3.1	功能描述	35
3.3.2	API 参考	40

1 概述

ROCKIVA SDK 是一套智能视频分析（IVA: Intelligence Video Analysis）算法 SDK，能够嵌入 NVR、IPC 等摄像头视频流的产品应用中，提供 AI 智能算法支持。SDK 的主要功能包括目标检测、周界功能、人脸抓拍等：

>目标检测功能：检测画面中的人形、人脸、机动车、非机动车和宠物猫狗，返回目标位置；

>周界功能：支持区域入侵、越界检测、区域进入、区域离开等周界功能 ；

2 SDK 模型说明

2.1 检测模型说明

为了满足多种产品类型的需求，SDK 提供了多个目标检测模型，用户可以根据需要选用。

可以直接配置 RockIvaInitParam 的 detModelName 为目标检测模型文件名，框架会自动从配置的模型目录路径下加载对应的模型文件。

1) 多合一检测模型

检测类别：人形、人脸、机动车、非机动车、车牌、宠物、人头

检测模型文件	适合分辨率	内存占用 MB	Flash 占用 MB (未压缩)	耗时 ms
iva_object_detection_v3_cls8_896x512.data	960x540	9.26	4.8	107
iva_object_detection_v3_cls8_640x384.data	640x360	7.11	4.8	67

2) 脸人宠

检测类别：人形、人脸、宠物

检测模型	适合分辨率	内存占用 MB	Flash 占用 MB (未压缩)	耗时 ms
iva_object_detection_v3_pfp_nn_896x512.data	960x540	7.71	3.2	79
iva_object_detection_v3_pfp_nn_640x384.data	640x360	5.57	3.2	52
iva_object_detection_v3_pfp_lite_896x512.data	960x540	5.73	1.1	91
iva_object_detection_v3_pfp_lite_640x384.data	640x360	4.16	1.1	58

3) 人车宠

检测类别：人形、人脸、机动车、宠物

检测模型	适合分	内存占用	Flash 占	耗时 ms
------	-----	------	---------	-------

	分辨率	MB	用 MB（未压缩）	
iva_object_detection_v3_pfcv_nn_896x512.data	960x540	7.71	3.2	79
iva_object_detection_v3_pfcv_nn_640x384.data	640x360	5.57	3.2	52
iva_object_detection_v3_pfcv_lite_896x512.data	960x540	5.73	1.1	91
iva_object_detection_v3_pfcv_lite_640x384.data	640x360	4.16	1.1	58

5) 人车

检测类别：人形、机动车

检测模型	适合分辨率	内存占用 MB	Flash 占用 MB（未压缩）	耗时 ms
iva_object_detection_v3_pv_lite_896x512.data	960x540	5.73	1.1	91
iva_object_detection_v3_pv_lite_640x384.data	640x360	4.16	1.1	58

3 SDK 开发说明

ROCKIVA SDK 首先基于目标检测跟踪算法从图像中获取人形、人脸、机动车、非机动车等目标信息，然后根据目标检测跟踪的结果再进行智能分析。当前支持的智能分析功能有周界、人脸等。智能分析每个功能可以独立开启或关闭，也能够同时使用。

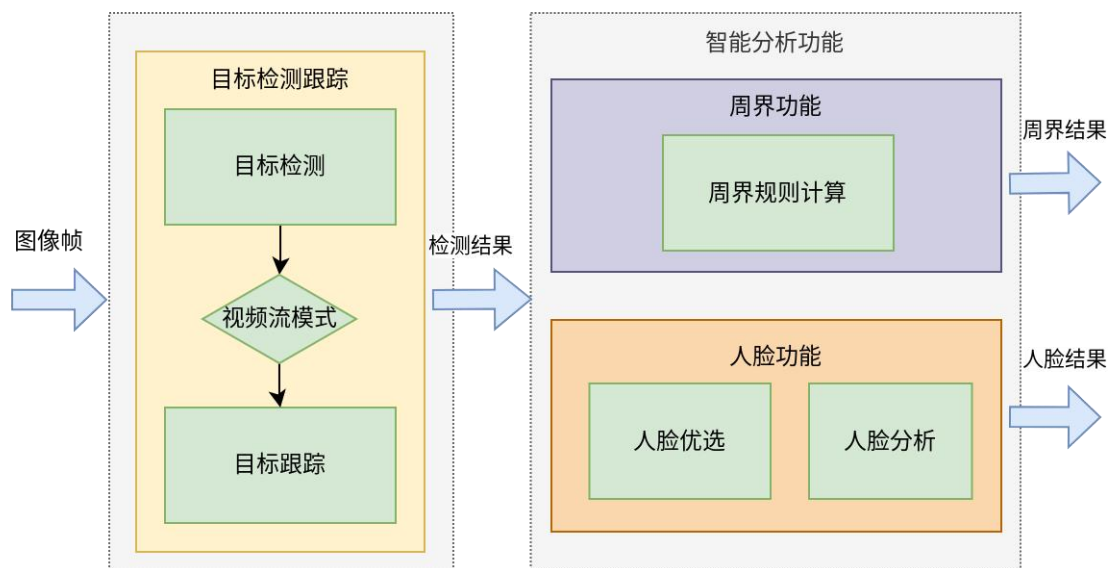


图 3-1 RovkIva SDK 整体流程

3.1 通用模块

3.1.1 功能描述

ROCKIVA SDK 的接口为异步模式，使用的基本操作就是配置初始化，然后按一定的帧率推帧，在回调函数中获取结果并释放图像帧内存，最后如果不需要使用就释放实例。

SDK 的基本调用流程如图 3-2 所示。

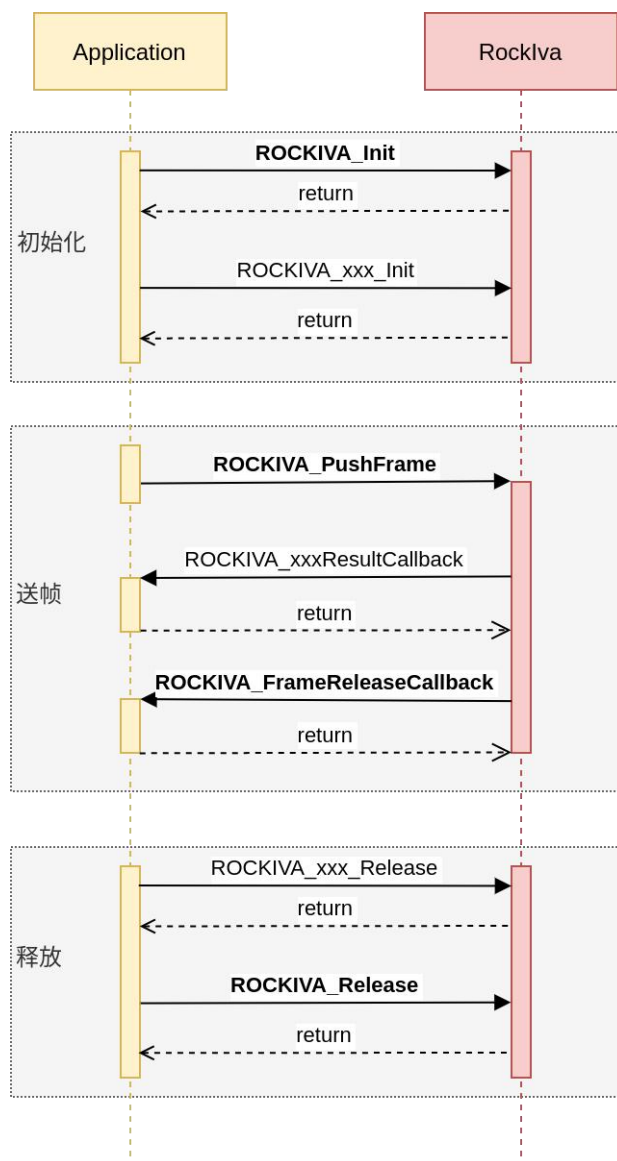


图 3-2 SDK 基本调用流程

配置初始化后，SDK 内部会创建线程异步处理每一帧，应用每次推送的图像帧都会放到一个

缓存队列中送给处理线程。

回调函数 `ROCKIVA_FrameReleaseCallback` 用来通知应用可以释放的图像帧，SDK 内部会判断如果不需要再使用该图像帧的时候通知外面应用可以进行释放。

调用代码参考：

```
RockIvaRetCode ret;
RockIvaHandle handle;

// 释放回调函数
void FrameReleaseCallback(const RockIvaReleaseFrames* releaseFrames, void* userdata)
{
    // 处理释放
}

// 模块回调函数，如检测模块
void DetResultCallback(const RockIvaDetectResult* result, const RockIvaExecuteStatus
status, void* userdata)
{
    // 处理结果
}

// 初始化 IVA 实例
RockIvaInitParam commonParams;
memset(&commonParams, 0, sizeof(RockIvaInitParam));
commonParams.detModelName = iva_object_detection_v3_cls8_896x512.data; // 设置检测模型
ret = ROCKIVA_Init(&handle, ROCKIVA_MODE_VIDEO, &commonParams, NULL);
// 设置回调函数
ROCKIVA_SetFrameReleaseCallback(handle, framerelease_callback);

// 初始化需要的 IVA 模块，如检测模块
ret = ROCKIVA_DETECT_Init(handle, &detParams, DetResultCallback);

// 处理图像帧
while(running) {
    RockIvaImage image;
    memset(&image, 0, sizeof(RockIvaImage));
    // 取帧
    ...
    // 设置帧数据
    image->info.width = image_width;
    image->info.height = image_height;
    image->info.format = image_pixel_format;
    image->dataAddr = image_buffer_virtual_address;
    image->dataFd = image_buffer_fd;
    image->frameId = frame_index;
    ...
    // 送帧
```



```
    ROCKIVA_PushFrame(handle, image, NULL);  
}  
  
// 等待所有帧都处理完成  
usleep(5*1000*1000);  
// 销毁模块  
ROCKIVA_DETECT_Release(handle)  
// 销毁 IVA 实例  
ROCKIVA_Release(handle);
```

3.1.2 API 参考

3.1.2.1 接口

接口	描述
ROCKIVA_Init	算法 SDK 初始化配置
ROCKIVA_Release	算法 SDK 销毁释放
ROCKIVA_WaitFinish	等待算法帧处理完成
ROCKIVA_SetFrameReleaseCallback	设置帧释放回调函数
ROCKIVA_FrameReleaseCallback	帧释放回调函数
ROCKIVA_PushFrame	输入图像帧
ROCKIVA_PushFrameWithRoi	输入图像帧的特定有效区域
ROCKIVA_GetVersion	获取 SDK 版本

3.1.2.1.1 ROCKIVA_Init

【功能】

算法 SDK 初始化，对传入 handle 进行初始化

【声明】

```
RockIvaRetCode ROCKIVA_Init(RockIvaHandle* handle, RockIvaWorkMode mode,  
                             const RockIvaInitParam* param, void *userdata);
```

【参数】

参数名称	输入/输出	描述
handle	输入	要初始化的实例句柄
mode	输入	工作模式，参见 RockIvaWorkMode 定义
param	输入	初始化参数，参见 RockIvaInitParam 定义
userdata	输入	用户自定义数据

【返回值】

RockIvaRetCode

3.1.2.1.2 ROCKIVA_Release

【功能】

算法 SDK 销毁释放

【声明】

```
RockIvaRetCode ROCKIVA_Release(RockIvaHandle handle);
```

【参数】

参数名称	输入/输出	描述
handle	输入	要释放的实例句柄

【返回值】

RockIvaRetCode

3.1.2.1.3 ROCKIVA_SetFrameReleaseCallback

【功能】

设置帧释放回调函数

【声明】

```
RockIvaRetCode ROCKIVA_SetFrameReleaseCallback(RockIvaHandle handle,  
ROCKIVA_FrameReleaseCallback callback);
```

【参数】

参数名称	输入/输出	描述
handle	输入	实例句柄
callback	输入	帧释放回调函数

【返回值】

RockIvaRetCode

【说明】

建议将帧对应的信息存放到 RockIvaImage 中的 extData 指针，然后释放时候可以取出进行释放

3.1.2.1.4 ROCKIVA_FrameReleaseCallback

【功能】

帧释放回调函数

【声明】

```
typedef void (*ROCKIVA_FrameReleaseCallback)(const RockIvaReleaseFrames* releaseFrames,  
void *userdata);
```

【参数】

参数名称	输入/输出	描述
releaseFrames	输入	可释放的帧列表，参见 RockIvaReleaseFrames 定义
userdata	输入	用户自定义数据

【返回值】

RockIvaRetCode

3.1.2.1.5 ROCKIVA_PushFrame

【功能】

输入图像帧

【声明】

```
RockIvaRetCode ROCKIVA_PushFrame(RockIvaHandle handle, const RockIvaImage* inputImg, const  
RockIvaFrameExtraInfo* extraInfo);
```

【参数】

参数名称	输入/输出	描述
handle	输入	实例句柄
inputImg	输入	输入图像帧，参见 RockIvaImage 定义
extraInfo	输入	额外信息(不需要可设 NULL)，参见 RockIvaFrameExtraInfo 定义

【返回值】

RockIvaRetCode

3.1.2.1.6 ROCKIVA_SetParam

【功能】

设置参数

【声明】

```
RockIvaRetCode ROCKIVA_SetParam(RockIvaParams* params, const char* key, const char* value);
```

【参数】

参数名称	输入/输出	描述
params	输入	参数, 参见 RockIvaParams 定义
key	输入	关键字
value	输入	值

【返回值】

RockIvaRetCode

3.1.2.1.7 ROCKIVA_GetVersion

【功能】

获取 SDK 版本号

【声明】

```
RockIvaRetCode ROCKIVA_GetVersion(const uint32_t maxLen, char* version);
```

【参数】

参数名称	输入/输出	描述
maxLen	输入	存放版本号 buffer 大小
version	输入	版本号 buffer 地址

【返回值】

RockIvaRetCode

3.1.2.1.8 ROCKIVA_Yuv2Jpeg

【功能】

回调函数, 用于将图像编码为 JPEG 图像

【声明】

```
int (*ROCKIVA_Yuv2Jpeg)(RockIvaImage *inputImg, RockIvaImage *outputImg, void *userdata);
```

【参数】

参数名称	输入/输出	描述
inputImg	输入	要编码的图像
outputImg	输出	编码后图像 (注意, outputImg.dataAddr 需要外部自己申请和释放内存, 当内部对该图像内存不再使用将会在抓拍回调函数 ROCKIVA_FrameReleaseCallback 上报)
userdata	输入	初始化设置的自定义指针

【返回值】

RockIvaRetCode

3.1.2.2 枚举

枚举	描述
RockIvaLogLevel	日志打印级别
RockIvaRetCode	函数返回错误码
RockIvaExecuteStatus	执行回调结果状态码
RockIvaImageFormat	图像像素格式
RockIvaImageTransform	输入图像的旋转模式
RockIvaMemType	内存类型
RockIvaObjectType	目标类型
RockIvaObjectState	目标跟踪状态
RockIvaDetModel	前级目标检测算法模型
RockIvaWorkMode	工作模式
RockIvaCameraType	摄像头类型

3.1.2.2.1 RockIvaLogLevel

【功能】

日志打印级别

【声明】

```
typedef enum {  
    ROCKIVA_LOG_ERROR = 0,  
    ROCKIVA_LOG_WARN = 1,  
    ROCKIVA_LOG_DEBUG = 2,  
    ROCKIVA_LOG_INFO = 3,  
}
```

```
    ROCKIVA_LOG_TRACE = 4  
} RockIvaLogLevel;
```

【成员】

成员	描述
ROCKIVA_LOG_ERROR	错误级别（默认）
ROCKIVA_LOG_WARN	警告级别
ROCKIVA_LOG_DEBUG	调试级别
ROCKIVA_LOG_INFO	信息级别
ROCKIVA_LOG_TRACE	跟踪调用级别

【说明】

日志等级数值越大，打印的日志越多。

3.1.2.2.2 RockIvaRetCode**【功能】**

函数调用返回错误码

【声明】

```
typedef enum {  
    ROCKIVA_RET_SUCCESS = 0,  
    ROCKIVA_RET_FAIL = -1,  
    ROCKIVA_RET_NULL_PTR = -2,  
    ROCKIVA_RET_INVALID_HANDLE = -3,  
    ROCKIVA_RET_LICENSE_ERROR = -4,  
    ROCKIVA_RET_UNSUPPORTED = -5,  
    ROCKIVA_RET_STREAM_SWITCH = -6,  
    ROCKIVA_RET_BUFFER_FULL = -7,  
} RockIvaRetCode;
```

【成员】

成员	描述
ROCKIVA_RET_SUCCESS	成功
ROCKIVA_RET_FAIL	失败
ROCKIVA_RET_NULL_PTR	空指针
ROCKIVA_RET_INVALID_HANDLE	无效句柄
ROCKIVA_RET_LICENSE_ERROR	License 错误
ROCKIVA_RET_UNSUPPORTED	不支持
ROCKIVA_RET_STREAM_SWITCH	码流切换
ROCKIVA_RET_BUFFER_FULL	缓存区域满

3.1.2.2.3 RockIvaExecuteStatus

【功能】

执行回调结果状态码

【声明】

```
typedef enum {
    ROCKIVA_SUCCESS = 0,
    ROCKIVA_UNKNOWN = 1,
    ROCKIVA_NULL_PTR = 2,
    ROCKIVA_ALLOC_FAILED = 3,
    ROCKIVA_INVALID_INPUT = 4,
    ROCKIVA_EXECUTE_FAILED = 6,
    ROCKIVA_NOT_CONFIGURED = 7,
    ROCKIVA_NO_CAPACITY = 8,
    ROCKIVA_BUFFER_FULL = 9,
    ROCKIVA_LICENSE_ERROR = 10,
    ROCKIVA_JPEG_DECODE_ERROR = 11,
    ROCKIVA_DECODER_EXIT = 12,
} RockIvaExecuteStatus;
```

【成员】

成员	描述
ROCKIVA_SUCCESS	运行结果正常
ROCKIVA_UNKNOWN	错误类型未知
ROCKIVA_NULL_PTR	操作空指针
ROCKIVA_ALLOC_FAILED	申请空间失败
ROCKIVA_INVALID_INPUT	无效的输入
ROCKIVA_EXECUTE_FAILED	内部执行错误
ROCKIVA_NOT_CONFIGURED	未配置的类型
ROCKIVA_NO_CAPACITY	业务已建满
ROCKIVA_BUFFER_FULL	缓存区域满
ROCKIVA_LICENSE_ERROR	license 异常
ROCKIVA_JPEG_DECODE_ERROR	解码出错
ROCKIVA_DECODER_EXIT	解码器内部退出

3.1.2.2.4 RockIvaImageFormat

【功能】

图像像素格式类型

【声明】

```
typedef enum {  
    ROCKIVA_IMAGE_FORMAT_GRAY8 = 0,  
    ROCKIVA_IMAGE_FORMAT_RGB888,  
    ROCKIVA_IMAGE_FORMAT_BGR888,  
    ROCKIVA_IMAGE_FORMAT_RGBA8888,  
    ROCKIVA_IMAGE_FORMAT_BGRA8888,  
    ROCKIVA_IMAGE_FORMAT_YUV420P_YU12,  
    ROCKIVA_IMAGE_FORMAT_YUV420P_YV12,  
    ROCKIVA_IMAGE_FORMAT_YUV420SP_NV12,  
    ROCKIVA_IMAGE_FORMAT_YUV420SP_NV21,  
    ROCKIVA_IMAGE_FORMAT_JPEG,  
} RockIvaImageFormat;
```

【成员】

成员	描述
ROCKIVA_IMAGE_FORMAT_GRAY8	单通道灰度图
ROCKIVA_IMAGE_FORMAT_RGB888	三通道彩色图像
ROCKIVA_IMAGE_FORMAT_BGR888	三通道彩色图像
ROCKIVA_IMAGE_FORMAT_RGBA8888	四通道彩色图像
ROCKIVA_IMAGE_FORMAT_BGRA8888	四通道彩色图像
ROCKIVA_IMAGE_FORMAT_YUV420P_YU12	YUV420P_YU12
ROCKIVA_IMAGE_FORMAT_YUV420P_YV12	YUV420P_YV12
ROCKIVA_IMAGE_FORMAT_YUV420SP_NV12	YUV420SP_NV12
ROCKIVA_IMAGE_FORMAT_YUV420SP_NV21	YUV420SP_NV21
ROCKIVA_IMAGE_FORMAT_JPEG	JPEG 图像（输入图像不支持该格式）

【注意事项】

无

3.1.2.2.5 RockIvaImageTransform**【功能】**

输入图像的旋转模式

【声明】

```
typedef enum {  
    ROCKIVA_IMAGE_TRANSFORM_NONE = 0x00,  
    ROCKIVA_IMAGE_TRANSFORM_FLIP_H = 0x01,  
    ROCKIVA_IMAGE_TRANSFORM_FLIP_V = 0x02,  
    ROCKIVA_IMAGE_TRANSFORM_ROTATE_90 = 0x04,  
    ROCKIVA_IMAGE_TRANSFORM_ROTATE_180 = 0x03,  
    ROCKIVA_IMAGE_TRANSFORM_ROTATE_270 = 0x07,  
} RockIvaImageFormat;
```

【成员】

成员	描述
ROCKIVA_IMAGE_TRANSFORM_NONE	正常
ROCKIVA_IMAGE_TRANSFORM_FLIP_H	水平翻转
ROCKIVA_IMAGE_TRANSFORM_FLIP_V	垂直翻转
ROCKIVA_IMAGE_TRANSFORM_ROTATE_90	顺时针 90 度
ROCKIVA_IMAGE_TRANSFORM_ROTATE_180	顺时针 180 度
ROCKIVA_IMAGE_TRANSFORM_ROTATE_270	顺时针 270 度

3.1.2.2.6 RockIvaObjectType**【功能】**

检测的目标类别

【声明】

```
typedef enum {  
    ROCKIVA_OBJECT_TYPE_NONE = 0,  
    ROCKIVA_OBJECT_TYPE_PERSON = 1,  
    ROCKIVA_OBJECT_TYPE_VEHICLE = 2,  
    ROCKIVA_OBJECT_TYPE_NON_VEHICLE = 3,  
    ROCKIVA_OBJECT_TYPE_FACE = 4,  
    ROCKIVA_OBJECT_TYPE_HEAD = 5,  
    ROCKIVA_OBJECT_TYPE_PET = 6,  
    ROCKIVA_OBJECT_TYPE_MOTORCYCLE = 7,  
    ROCKIVA_OBJECT_TYPE_BICYCLE = 8,  
    ROCKIVA_OBJECT_TYPE_PLATE = 9,  
    ROCKIVA_OBJECT_TYPE_BABY = 10,  
    ROCKIVA_OBJECT_TYPE_MAX  
} RockIvaObjectType;
```

【成员】

成员	描述
ROCKIVA_OBJECT_TYPE_NONE	未知
ROCKIVA_OBJECT_TYPE_PERSON	人形
ROCKIVA_OBJECT_TYPE_VEHICLE	机动车
ROCKIVA_OBJECT_TYPE_NON_VEHICLE	非机动车
ROCKIVA_OBJECT_TYPE_FACE	人脸
ROCKIVA_OBJECT_TYPE_HEAD	人头
ROCKIVA_OBJECT_TYPE_PET	猫、狗
ROCKIVA_OBJECT_TYPE_MOTORCYCLE	电瓶车
ROCKIVA_OBJECT_TYPE_BICYCLE	自行车
ROCKIVA_OBJECT_TYPE_PLATE	车牌
ROCKIVA_OBJECT_TYPE_BABY	婴幼儿

3.1.2.2.7 RockIvaObjectState

【功能】

目标跟踪状态

【声明】

```
typedef enum {  
    ROCKIVA_OBJECT_STATE_NONE,  
    ROCKIVA_OBJECT_STATE_FIRST,  
    ROCKIVA_OBJECT_STATE_TRACKING,  
    ROCKIVA_OBJECT_STATE_LOST,
```

```
    ROCKIVA_OBJECT_STATE_DISPEAR,  
} RockIvaObjectState;
```

【成员】

成员	描述
ROCKIVA_OBJECT_STATE_NONE	目标状态未知
ROCKIVA_OBJECT_STATE_FIRST	目标第一次出现
ROCKIVA_OBJECT_STATE_TRACKING	目标检测跟踪过程中
ROCKIVA_OBJECT_STATE_LOST	目标检测丢失
ROCKIVA_OBJECT_STATE_DISPEAR	目标消失

3.1.2.2.8 RockIvaDetModel**【功能】**

前级目标检测算法模型

【声明】

```
typedef enum {  
    ROCKIVA_DET_MODEL_NONE = 0,  
    ROCKIVA_DET_MODEL_CLS7,  
    ROCKIVA_DET_MODEL_PFCP,  
    ROCKIVA_DET_MODEL_PFP,  
    ROCKIVA_DET_MODEL_PHS,  
    ROCKIVA_DET_MODEL_PHCP,  
    ROCKIVA_DET_MODEL_PERSON,  
    ROCKIVA_DET_MODEL_NONVEHICLE,  
    ROCKIVA_DET_MODEL_FHS,  
    ROCKIVA_DET_MODEL_PHCN,  
    ROCKIVA_DET_MODEL_CLS8,  
} RockIvaDetModel;
```

【成员】

成员	描述
ROCKIVA_DET_MODEL_NONE	未设置，不需要前级目标检测
ROCKIVA_DET_MODEL_CLS7	模型文件：object_detection_ipc_cls7.data 检测类别：人形、人脸、机动车、车牌、非机动车、宠物
ROCKIVA_DET_MODEL_PFCP	模型文件：object_detection_pfcps.data 检测类别：人形、人脸、机动车、宠物
ROCKIVA_DET_MODEL_PFP	模型文件：object_detection_pfps.data 检测类别：人形、人脸、宠物
ROCKIVA_DET_MODEL_PHS	模型文件：object_detection_phs.data 检测类别：人形、头肩
ROCKIVA_DET_MODEL_PHCP	模型文件：object_detection_phcps.data 检测类别：人形、头肩、机动车、宠物
ROCKIVA_DET_MODEL_PERSON	模型文件：object_detection_person.data 检测类别：人形
ROCKIVA_DET_MODEL_NONVEHICLE	模型文件：object_detection_motorcycle.data 检测类别：电动车
ROCKIVA_DET_MODEL_FHS	模型文件：object_detection_fhs.data 检测类别：人脸、头肩
ROCKIVA_DET_MODEL_PHCN	模型文件：object_detection_phcns.data 检测类别：人形、头肩、机动车、非机动车
ROCKIVA_DET_MODEL_CLS8	模型文件：object_detection_v3_cls8.data 检测类别：人形、人脸、机动车、车牌、非机动车、宠物、人头 注：相比CLS7 速度更快，但内存会略微增加

【说明】

3.1.2.2.9 RockIvaWorkMode

【功能】

工作模式

【声明】

```
typedef enum {
    ROCKIVA_MODE_VIDEO    = 0,
    ROCKIVA_MODE_PICTURE  = 1,
} RockIvaWorkMode;
```

【成员】

成员	描述
ROCKIVA_MODE_VIDEO	视频流模式 (使能跟踪)
ROCKIVA_MODE_PICTURE	图片模式 (不使能跟踪)

【说明】

周界功能必须要工作在视频流模式；检测和人脸功能可以使用视频流或图片模式；

3.1.2.2.10 RockIvaMemType

【功能】

内存类型

【声明】

```
typedef enum {  
    ROCKIVA_MEM_TYPE_CPU = 0,  
    ROCKIVA_MEM_TYPE_DMA = 1  
} RockIvaMemType;
```

【成员】

成员	描述
ROCKIVA_MEM_TYPE_CPU	虚拟地址内存
ROCKIVA_MEM_TYPE_DMA	DMA BUF 内存（RV1106）

【说明】

为图像分配的缓冲区内存类型，对于 RV1106 平台 RGA 需要使用 DMA BUF 内存类型。

3.1.2.2.11 RockIvaCameraType

【功能】

摄像头类型

【声明】

```
typedef enum {  
    ROCKIVA_CAMERA_TYPE_ONE = 0,  
    ROCKIVA_CAMERA_TYPE_DUAL = 1,  
    ROCKIVA_CAMERA_TYPE_SL = 2,  
} RockIvaCameraType;
```

【成员】

成员	描述
ROCKIVA_CAMERA_TYPE_ONE	单摄
ROCKIVA_CAMERA_TYPE_DUAL	双摄，能够提供 RGB 和 IR 两路图像
ROCKIVA_CAMERA_TYPE_SL	结构光摄像头，能够提供 RGB、IR 和 Depth 三路图像

3.1.2.3 结构体

结构体	描述
RockIvaPoint	点坐标
RockIvaLine	线坐标
RockIvaRectangle	正四边形坐标
RockIvaQuadrangle	四边形坐标
RockIvaArea	多边形区域
RockIvaAreas	多边形区域列表
RockIvaSize	目标大小
RockIvaAngle	角度信息
RockIvaRectExpandRatio	检测框向四周扩展的比例大小配置
RockIvaImageInfo	图像信息
RockIvaImage	图像
RockIvaObjectInfo	单个目标检测结果信息
RockIvaDetectResult	目标检测结果
RockIvaReleaseFrames	需要释放的帧列表
RockIvaInitParam	SDK 全局参数配置

3.1.2.3.1 RockIvaPoint

【功能】

点坐标

【声明】

```
typedef struct {
    uint16_t x;
    uint16_t y;
} RockIvaPoint;
```

【成员】

成员	描述
x	横坐标，万分比表示，0~9999 有效
y	纵坐标，万分比表示，0~9999 有效

3.1.2.3.2 RockIvaLine

【功能】

线坐标

【声明】

```
typedef struct {
    RockIvaPoint head;
    RockIvaPoint tail;
} RockIvaLine;
```

【成员】

成员	描述
head	头坐标
tail	尾坐标

3.1.2.3.3 RockIvaRectangle

【功能】

正四边形坐标

【声明】

```
typedef struct {
    RockIvaPoint topLeft;
    RockIvaPoint bottomRight;
} RockIvaRectangle;
```

【成员】

成员	描述
topLeft	左上角坐标
bottomRight	右下角坐标

3.1.2.3.4 RockIvaQuadrangle

【功能】

任意四边形坐标

【声明】

```
typedef struct {  
    RockIvaPoint topLeft;  
    RockIvaPoint topRight;  
    RockIvaPoint bottomLeft;  
    RockIvaPoint bottomRight;  
} RockIvaQuadrangle;
```

【成员】

成员	描述
topLeft	左上角坐标
topRight	右上角坐标
bottomLeft	左下角坐标
bottomRight	右下角坐标

3.1.2.3.5 RockIvaArea

【功能】

多边形区域坐标

【声明】

```
typedef struct {  
    uint32_t pointNum;  
    RockIvaPoint points[ROCKIVA_AREA_POINT_NUM_MAX];  
} RockIvaArea;
```

【成员】

成员	描述
pointNum	点个数
points	围成区域的所有点坐标

3.1.2.3.6 RockIvaAreas

【功能】

多区域

【声明】


```
typedef struct {
    uint32_t areaNum;

    RockIvaArea areas[ROCKIVA_AREA_NUM_MAX];
} RockIvaAreas;
```

【成员】

成员	描述
areaNum	区域数量
areas	区域数组

3.1.2.3.7 RockIvaSize**【功能】**

目标大小向四周

【声明】

```
typedef struct {
    uint16_t width;
    uint16_t height;
} RockIvaSize;
```

【成员】

成员	描述
width	宽度（万分比坐标，值范围 0~9999）
height	高度（万分比坐标，值范围 0~9999）

3.1.2.3.8 RockIvaRectExpandRatio**【功能】**

检测框向四周扩展的比例大小配置

【声明】

```
typedef struct {
    float up;
    float down;
    float left;
    float right;
} RockIvaRectExpandRatio;
```

【成员】

成员	描述
up	检测框向上按框高扩展的比例大小
down	检测框向下按框高扩展的比例大小
left	检测框向左按框宽扩展的比例大小
right	检测框向右按框宽扩展的比例大小

3.1.2.3.9 RockIvaAngle

【功能】

人脸角度信息

【声明】

```
typedef struct {  
    int16_t pitch;  
    int16_t yaw;  
    int16_t roll;  
} RockIvaAngle;
```

【成员】

成员	描述
pitch	俯仰角, 表示绕 x 轴旋转角度
yaw	偏航角, 表示绕 y 轴旋转角度
roll	翻滚角, 表示绕 z 轴旋转角度

3.1.2.3.10 RockIvaImageInfo

【功能】

图像信息

【声明】

```
typedef struct {  
    uint16_t width;  
    uint16_t height;  
    RockIvaImageFormat format;  
    RockIvaImageTransform transformMode;  
} RockIvaImageInfo;
```

【成员】

成员	描述
width	图像宽度
height	图像高度
format	图像像素格式
transformMode	旋转模式

3.1.2.3.11 RockIvaImage

【功能】

图像

【声明】

```
typedef struct {
    uint32_t frameId;
    uint32_t channelId;
    RockIvaImageInfo info;
    uint32_t size;
    uint8_t* dataAddr;
    uint8_t* dataPhyAddr;
    int32_t dataFd;
    void* extData;
} RockIvaImage;
```

【成员】

成员	描述
frameId	原始采集帧序号（应用自定义）
channelId	通道号（应用自定义，单通道置为 0）
info	图像信息
size	图像数据大小，图像格式为 JPEG 时需要
dataAddr	输入图像数据虚地址
dataPhyAddr	输入图像数据的物理地址（没有用则置为 NULL）
dataFd	输入图像数据的文件句柄（没有用则置为 0）
extData	用户自定义扩展数据

3.1.2.3.12 RockIvaObjectInfo

【功能】

单个目标检测结果信息

【声明】

```
typedef struct {
```

```
uint32_t objId;  
uint32_t frameId;  
uint32_t score;  
RockIvaRectangle rect;  
RockIvaObjectType type;  
} RockIvaObjectInfo;
```

【成员】

成员	描述
objId	目标 ID[0~2 ³²)
frameId	帧 ID
score	目标检测分数 [1-100]
rect	目标区域框 (万分比)
type	目标类别

3.1.2.3.13 RockIvaDetectResult**【功能】**

目标检测结果

【声明】

```
typedef struct {  
    uint32_t frameId;  
    RockIvaImage frame;  
    uint32_t channelId;  
    uint32_t objNum;  
    RockIvaObjectInfo objInfo[ROCKIVA_MAX_OBJ_NUM];  
} RockIvaDetectResult;
```

【成员】

成员	描述
frameId	帧 ID
frame	对应的输入图像帧
channelId	通道 ID
objNum	目标个数
objInfo	各目标检测信息

3.1.2.3.14 RockIvaReleaseFrames**【功能】**

可以释放的帧列表

【声明】

```
typedef struct {  
    uint32_t channelId;  
    uint32_t count;  
    RockIvaImage frameId[ROCKIVA_MAX_OBJ_NUM];  
} RockIvaReleaseFrames;
```

【成员】

成员	描述
channelId	通道 ID
count	数量
frameId	可释放的帧

3.1.2.3.15 RockIvaMediaOps

【功能】

多媒体操作函数

【声明】

```
typedef struct {  
    int (*ROCKIVA_Yuv2Jpeg)(RockIvaImage *inputImg, RockIvaImage *outputImg, void  
*userdata);  
} RockIvaMediaOps;
```

【成员】

成员	描述
ROCKIVA_Yuv2Jpeg	将图像编码为 JPEG 图像

3.1.2.3.16 RockIvaInitParam

【功能】

算法全局参数配置

【声明】

```
typedef struct {  
    RockIvaLogLevel logLevel;  
    char logPath[ROCKIVA_PATH_LENGTH];  
    char modelPath[ROCKIVA_PATH_LENGTH];  
}
```

```
RockIvaMemInfo license;

uint32_t coreMask;

uint32_t channelId;

RockIvaImageInfo imageInfo;

RockIvaAreas roiAreas;

RockIvaCameraType cameraType;

RockIvaDetModel detModel;

uint32_t detClasses;

RockIvaMemInfo detModelData;

uint8_t trackerVersion;

RockIvaMediaOps mediaOps;

} RockIvaInitParam;
```

【成员】

成员	描述
RockIvaLogLevel	日志等级
logPath	日志输出路径
modelPath	存放算法模型路径
license	License 信息
coreMask	指定使用哪个 NPU 核跑 (仅 RK3588 平台有效)
channelId	通道号
imageInfo	输入图像信息
roiAreas	有效区域
cameraType	摄像头类型
detModel	前级目标检测模型
detClasses	正常不需要配置，只有一些特殊检测类别需要配置，如婴儿检测： ROCKIVA_OBJECT_TYPE_BITMASK(ROCKIVA_OBJECT_TYPE_BABY)
detModelData	指定检测模型数据（用于快启直接配置模型内存数据直接用于加载）
detModelName	指定检测模型名字
trackerVersion	指定跟踪算法版本（废弃）
mediaOps	设置媒体操作回调函数

3.2 检测模块

3.2.1 功能描述

检测模型主要获取图像中检测到的目标的框坐标和分数等信息。检测模块对应头文件为 rockiva_det_api.h，可以通过 ROCKIVA_DETECT_Init 配置初始化参数并设置检测结果回调。基本调用流程如图 3-3 所示。

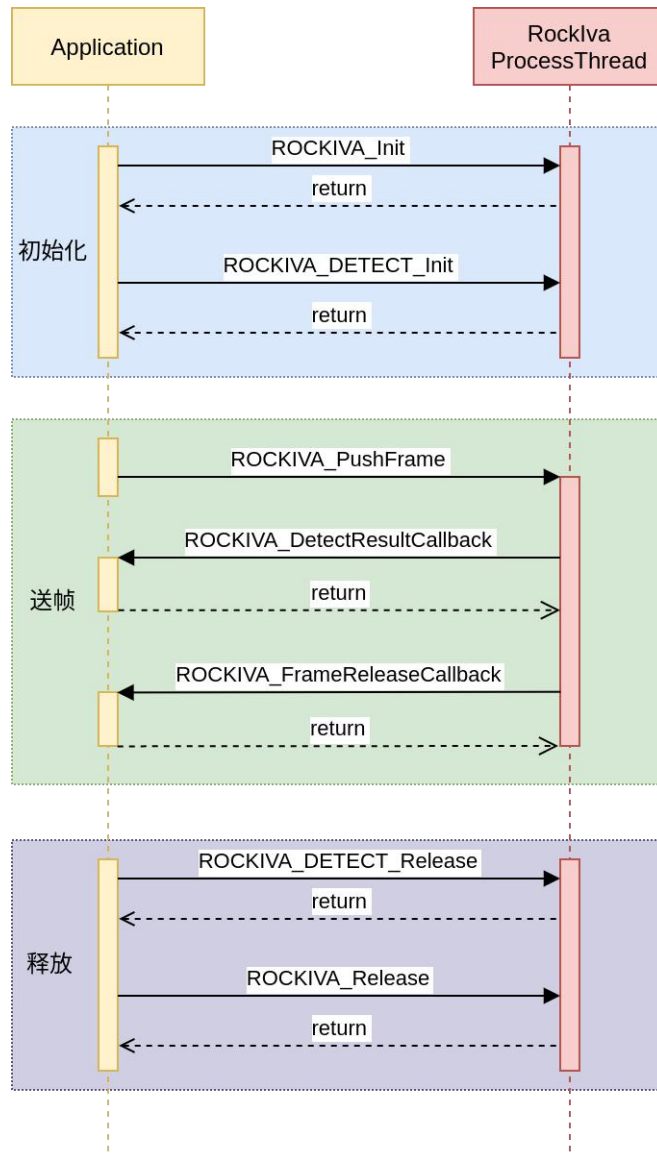


图 3-3 检测功能调用流程

配置初始化后，SDK 内部会创建线程异步处理每一帧，应用每次推送的图像帧都会放到一个缓存队列中送给处理线程。需要注意的是结果回调函数 `ROCKIVA_DetectResultCallback` 是运行在该内部处理线程，因此尽量不要做耗时太久的操作，否则会影响处理帧率。

检测模块可以工作在视频流模式或图片模式。视频流模式下会使能目标跟踪，检测回调中的每个检测目标的 `objId` 有效；图片模式下会关闭目标跟踪，检测目标的 `objId` 值无效。

对于用户推送进来的每一帧同样是通过 `ROCKIVA_FrameReleaseCallback` 回调函数返回的结果进行释放。

以下为检测模块的参考代码实例：

```
RockIvaRetCode ret;  
RockIvaHandle handle;
```

```
// 释放回调函数（同通用模块示例代码）
...

// 检测模块回调函数
void DetResultCallback(const RockIvaDetectResult* result, const RockIvaExecuteStatus
status, void* userdata)
{
    // 处理结果
}

// 初始化 IVA 实例
RockIvaInitParam commonParams;
memset(&commonParams, 0, sizeof(RockIvaInitParam));
commonParams.detModelName = iva_object_detection_v3_cls8_896x512.data; // 设置检测模型
vret = ROCKIVA_Init(&handle, ROCKIVA_MODE_VIDEO, &commonParams, NULL);
// 设置回调函数
ROCKIVA_SetFrameReleaseCallback(handle, framerelease_callback);

// 初始化检测模块
RockIvaDetTaskParams detParams;
memset(&detParams, 0, sizeof(RockIvaDetTaskParams));
// 设置上报目标类型
detParams->detObjectType |= ROCKIVA_OBJECT_TYPE_BITMASK(ROCKIVA_OBJECT_TYPE_PERSON)
// 设置检测分数阈值，只设置第 0 个可以对所有类别生效
detParams->scores[0] = 30;
ret = ROCKIVA_DETECT_Init(handle, &detParams, DetResultCallback);

// 送帧处理（同通用模块示例代码）
...

// 等待所有帧都处理完成
usleep(5*1000*1000);

// 销毁检测模块
ROCKIVA_DETECT_Release(handle)
// 销毁 IVA 实例
ROCKIVA_Release(handle);
```

3.2.2 API 参考

3.2.2.1 函数接口

接口	描述
ROCKIVA_DetectResultCallback	结果回调函数
ROCKIVA_DETECT_Init	初始化

ROCKIVA_DETECT_Release	释放
ROCKIVA_DETECT_Reset	运行时重新配置 (重新配置会导致内部的一些记录清空复位，但是模型不会重新初始化)

3.2.2.1.1 ROCKIVA_DetectResultCallback

【功能】

结果回调函数

【声明】

```
typedef void (*ROCKIVA_DetectResultCallback) (  
    const RockIvaDetectResult* result,  
    const RockIvaExecuteStatus status,  
    void* userdata);
```

【输入参数】

参数名称	输入/输出	描述
result	输入	结果
status	输入	状态码
userdata	输入	用户自定义数据

【返回值】

RockIvaRetCode

3.2.2.1.2 ROCKIVA_DETECT_Init

【功能】

初始化检测模块

【声明】

```
RockIvaRetCode ROCKIVA_DETECT_Init(RockIvaHandle handle,  
    const RockIvaDetTaskParams* params,  
    const ROCKIVA_DetectResultCallback resultCallback);
```

【输入参数】

参数名称	输入/输出	描述
handle	输入	实例句柄
params	输入	初始化参数
resultCallback	输入	结果回调函数

【返回值】

RockIvaRetCode

3.2.2.1.3 ROCKIVA_DETECT_Release**【功能】**

释放

【声明】

```
RockIvaRetCode ROCKIVA_DETECT_Release(RockIvaHandle handle);
```

【输入参数】

参数名称	输入/输出	描述
handle	输入	实例句柄

【返回值】

RockIvaRetCode

3.2.2.1.4 ROCKIVA_DETECT_Reset**【功能】**

运行时重新配置(重新配置会导致内部的一些记录清空复位，但是模型不会重新初始化)

【声明】

```
RockIvaRetCode ROCKIVA_DETECT_Reset(RockIvaHandle handle ,  
                                      RockIvaDetTaskParams* params);
```

【输入参数】

参数名称	输入/输出	描述
handle	输入	要重置的 handle，运行时重新配置 (重新配置会导致内部的一些记录清空复位，但是模型不会重新初始化)
params	输入	初始化参数

【返回值】

RockIvaRetCode

3.2.2.2 结构体

结构体	描述
RockIvaDetTaskParams	目标检测业务初始化参数配置

3.2.2.2.1 RockIvaDetTaskParams

【功能】

目标检测业务初始化参数配置

【声明】

```
typedef struct {
    uint32_t detObjectType;
    RockIvaAreas roiAreas;
    uint8_t scores[ROCKIVA_OBJECT_TYPE_MAX];
    uint8_t min_det_count;
} RockIvaDetTaskParams;
```

【成员】

成员	描述
detObjectType	配置要返回的检测目标
roiAreas	配置有效检测区域（仅影响检测结果）
scores	各类别过滤分数阈值，0 为内部自动
min_det_count	检测目标稳定检测帧数大于该数值后再上报，过滤一些小概率误检

3.3 周界模块

3.3.1 功能描述

周界功能的流程基本和检测类似，通过 ROCKIVA_BA_Init 初始化周界相关配置以及配置

周界结果回调。

周界功能模块代码调用流程如图 3-4 所示。

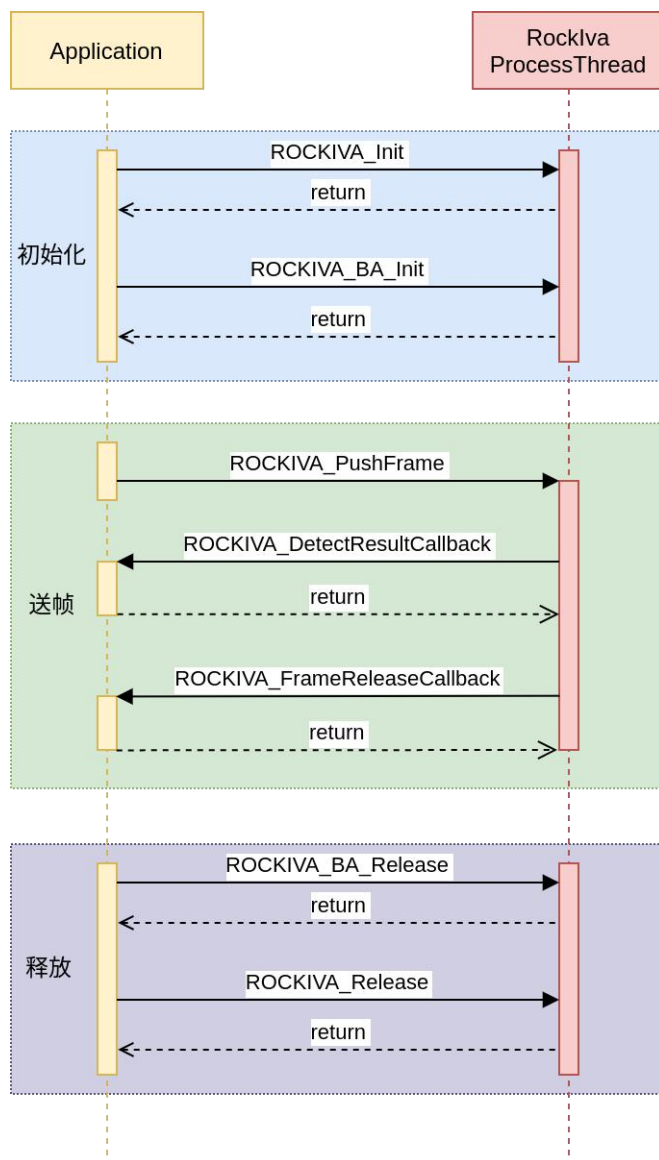


图 3-4 周界功能调用流程

注意周界的工作模式必须要是视频流模式（ROCKIVA_MODE_VIDEO），因为周界的规则判断需要根据目标前后帧的结果进行计算。

周界功能有四种类型：

- > 区域入侵：目标进入设定区域并停留超过设定时间触发
- > 区域进入：目标从设定区域外进入区域内触发
- > 区域离开：目标从设定区域内离开区外触发
- > 越界检测：目标从设定的线段越过（符合设定方向）触发

可配置规则主要有：

规则配置	说明
区域	设定规则区域：有顺序的最多 6 个点构成的多边形
界线	设定规则界线：两个点构成的线段，可设置方向
目标类型过滤	设定触发的目标类型：人形、车辆、非机动车
目标大小过滤	设定目标大小：最小和最大宽高

区域入侵结果展示如图 3-5 和图 3-6 所示。目标进入禁区后会上报事件。

越界结果展示如图 3-7 所示。目标由禁止的方向越过边界线时会上报事件。

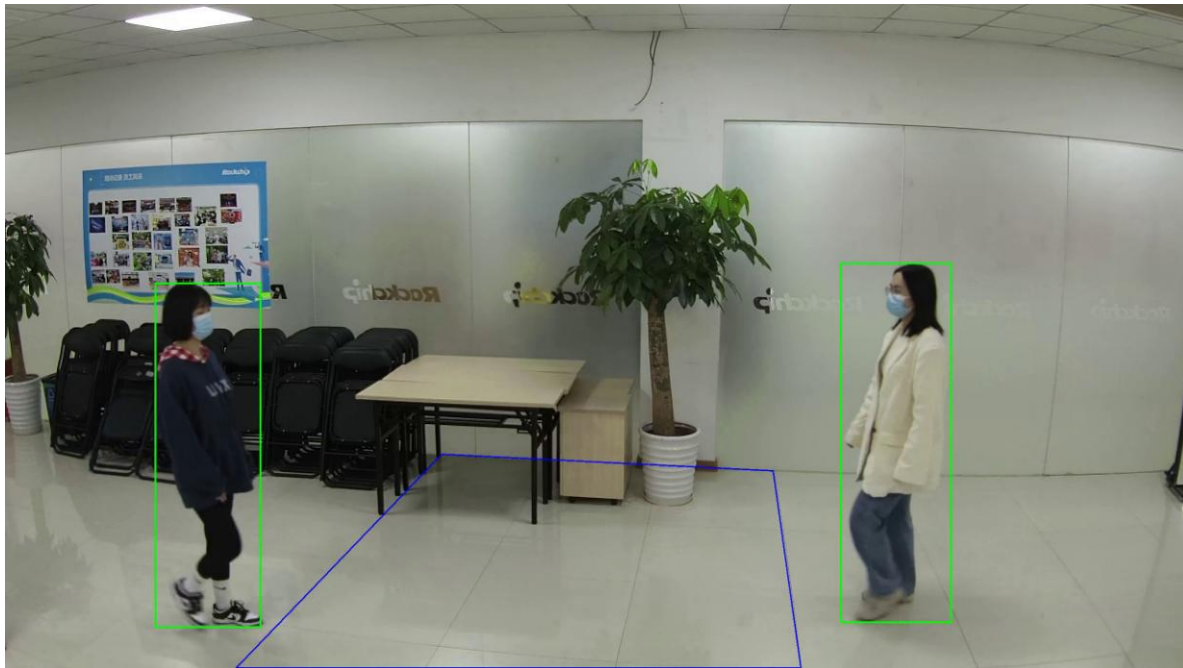


图 3-5 区域入侵进入禁区前

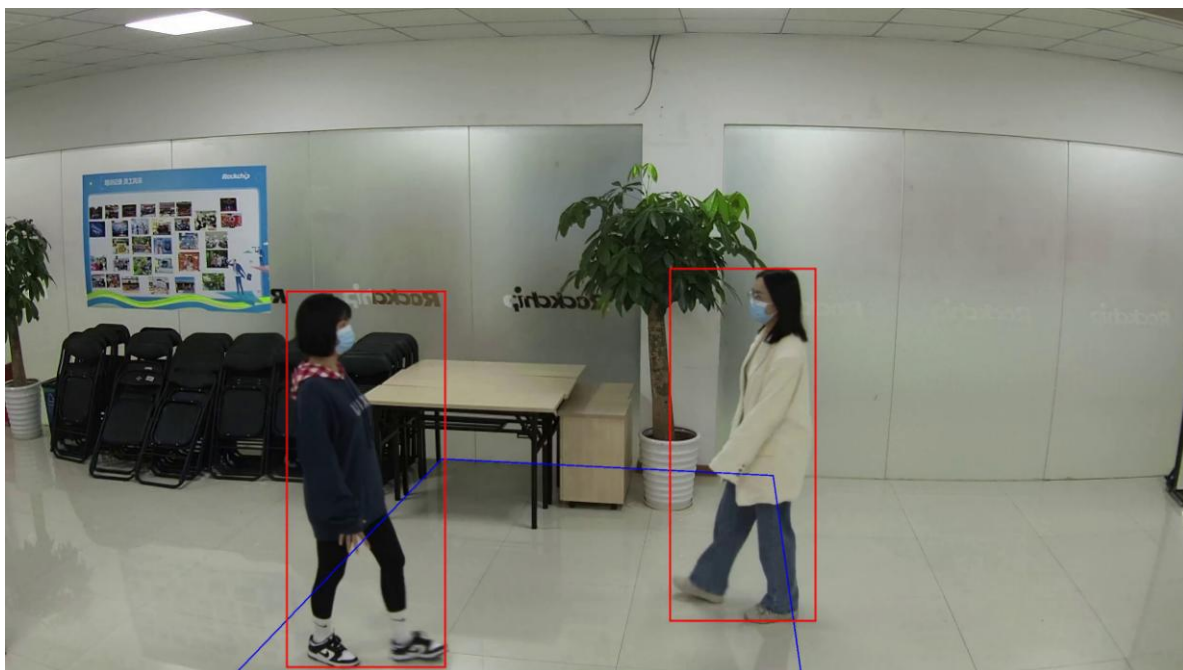


图 3-6 区域入侵进入禁区后

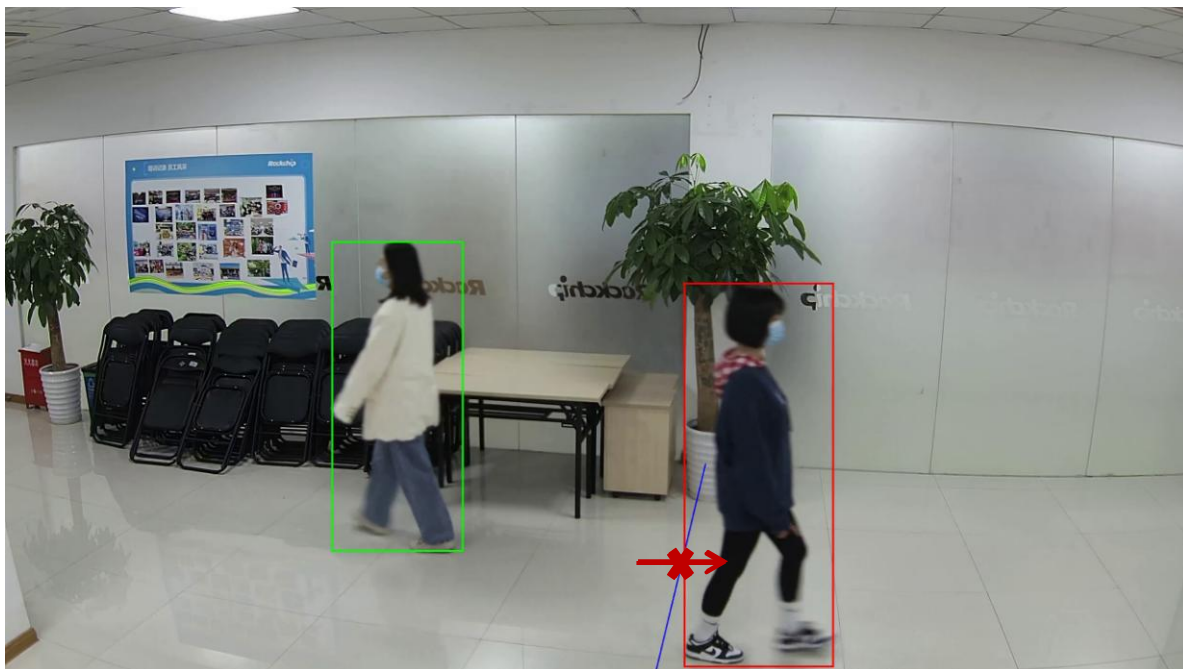


图 3-7 越界结果

以下为周界模块相关参考代码

```
RockIvaRetCode ret;
RockIvaHandle handle;

// 释放回调函数（同通用模块示例代码）
...

// 周界模块回调函数
void BaResultCallback(const RockIvaBaResult* result, const RockIvaExecuteStatus status,
void* userdata)
{
    // 处理结果
}

// 初始化 IVA 实例
RockIvaInitParam commonParams;
memset(&commonParams, 0, sizeof(RockIvaInitParam));
commonParams.detModelName = iva_object_detection_v3_cls8_896x512.data; // 设置检测模型

ret = ROCKIVA_Init(&handle, ROCKIVA_MODE_VIDEO, &commonParams, NULL);
// 设置回调函数
ROCKIVA_SetFrameReleaseCallback(handle, framerelease_callback);

// 初始化周界模块
RockIvaBaTaskParams baParams;
memset(&baParams, 0, sizeof(RockIvaBaTaskParams));
```

```
// 构建一个区域入侵规则

baParams.baRules.areaInBreakRule[0].ruleEnable = 1;
baParams.baRules.areaInBreakRule[0].alertTime = 1000; // 目标在区域内触发时间
baParams.baRules.areaInBreakRule[0].ruleID = 1;
baParams.baRules.areaInBreakRule[0].objType |=
    ROCKIVA_OBJECT_TYPE_BITMASK(ROCKIVA_OBJECT_TYPE_PERSON);
// 区域由多个点按顺序连成
baParams.baRules.areaInBreakRule[0].area.pointNum = 4;
baParams.baRules.areaInBreakRule[0].area.points[0].x = 1000;
baParams.baRules.areaInBreakRule[0].area.points[0].y = 1000;
baParams.baRules.areaInBreakRule[0].area.points[1].x = 9000;
baParams.baRules.areaInBreakRule[0].area.points[1].y = 1000;
baParams.baRules.areaInBreakRule[0].area.points[2].x = 9000;
baParams.baRules.areaInBreakRule[0].area.points[2].y = 9000;
baParams.baRules.areaInBreakRule[0].area.points[3].x = 1000;
baParams.baRules.areaInBreakRule[0].area.points[3].y = 9000;

// 构建一个越界规则
baParams.baRules.tripWireRule[0].ruleEnable = 1;
baParams.baRules.tripWireRule[0].event = ROCKIVA_BA_TRIP_EVENT_BOTH;
baParams.baRules.tripWireRule[0].ruleID = 2;
baParams.baRules.tripWireRule[0].objType |=
    ROCKIVA_OBJECT_TYPE_BITMASK(ROCKIVA_OBJECT_TYPE_PERSON);
baParams.baRules.tripWireRule[0].line.head.x = 1000;
baParams.baRules.tripWireRule[0].line.head.y = 5000;
baParams.baRules.tripWireRule[0].line.tail.x = 9000;
baParams.baRules.tripWireRule[0].line.tail.y = 5000;

ret = ROCKIVA_BA_Init(handle, &baParams, BaResultCallback);

// 送帧处理（同通用模块示例代码）
...

// 等待所有帧都处理完成
usleep(5*1000*1000);

// 销毁周界模块
ROCKIVA_BA_Release(handle)
// 销毁 IVA 实例
ROCKIVA_Release(handle);
```

3.3.2 API 参考

3.3.2.1 函数接口

接口	描述
ROCKIVA_BA_ResultCallback	结果回调函数
ROCKIVA_BA_Init	初始化
ROCKIVA_BA_Destroy	销毁
ROCKIVA_BA_Reset	重置

3.3.2.1.1 ROCKIVA_BA_ResultCallback

【功能】

结果回调函数

【声明】

```
typedef void(*ROCKIVA_BA_ResultCallback) (
    const RockIvaBaResult *result,
    const RockIvaExecuteStatus status,
    void *userdata);
```

【输入参数】

参数名称	输入/输出	描述
result	输入	结果
status	输入	状态码
userdata	输入	用户自定义数据

【返回值】

RockIvaRetCode

3.3.2.1.2 ROCKIVA_BA_Init

【功能】

初始化

【声明】

```
RockIvaRetCode ROCKIVA_BA_Init(RockIvaHandle handle,
    const RockIvaBaTaskParams *initParams,
```



```
const ROCKIVA_BA_ResultCallback resultCallback);
```

【输入参数】

参数名称	输入/输出	描述
handle	输入	要初始化的 handle
initParams	输入	初始化参数配置
resultCallback	输入	回调函数

【返回值】

RockIvaRetCode

3.3.2.1.3 ROCKIVA_BA_Destroy

【功能】

销毁

【声明】

```
RockIvaRetCode ROCKIVA_BA_Destroy(RockIvaHandle handle);
```

【输入参数】

参数名称	输入/输出	描述
handle	输入	要销毁的 handle

【返回值】

RockIvaRetCode

3.3.2.1.4 ROCKIVA_BA_Reset

【功能】

重置

【声明】

```
RockIvaRetCode ROCKIVA_BA_Reset(RockIvaHandle handle, const RockIvaBaTaskParams* params);
```

【输入参数】

参数名称	输入/输出	描述
handle	输入	要重置的 handle
params	输入	新的初始化参数配置

【返回值】

RockIvaRetCode

3.3.2.2 枚举定义

枚举	描述
RockIvaBaTripEvent	周界规则类型（绊线/区域事件）
RockIvaBaRuleTriggerType	目标规则触发类型

3.3.2.2.1 RockIvaBaTripEvent

【功能】

周界规则绊线事件类型

【声明】

```
typedef enum {  
    ROCKIVA_BA_TRIP_EVENT_BOTH = 0,  
    ROCKIVA_BA_TRIP_EVENT_DEASIL = 1,  
    ROCKIVA_BA_TRIP_EVENT_WIDDERSHINES = 2,  
} RockIvaBaTripEvent;
```

【成员】

成员	描述
ROCKIVA_BA_TRIP_EVENT_BOTH	绊线:双向触发
ROCKIVA_BA_TRIP_EVENT_DEASIL	绊线:顺时针触发
ROCKIVA_BA_TRIP_EVENT_WIDDERSHINES	绊线:逆时针触发

3.3.2.2.2 RockIvaBaRuleTriggerType

【功能】

目标规则触发类型

【声明】

```
typedef enum {  
    ROCKIVA_BA_RULE_NONE = 0,
```

```
ROCKIVA_BA_RULE_CROSS = 0x1,  
ROCKIVA_BA_RULE_INAREA = 0x2,  
ROCKIVA_BA_RULE_OUTAREA = 0x4,  
ROCKIVA_BA_RULE_STAY = 0x8,  
} RockIvaBaRuleTriggerType;
```

【成员】

成员	描述
ROCKIVA_BA_RULE_NONE	
ROCKIVA_BA_RULE_CROSS	拌线
ROCKIVA_BA_RULE_INAREA	进入区域
ROCKIVA_BA_RULE_OUTAREA	离开区域
ROCKIVA_BA_RULE_STAY	区域入侵

3.3.2.3 结构体定义

结构体	描述
RockIvaBaWireRule	越界规则
RockIvaBaAreaRule	区域规则
RockIvaBaTaskRule	行为分析类规则配置
RockIvaBaAiConfig	算法配置
RockIvaBaTaskParams	行为分析业务初始化参数配
RockIvaBaTrigger	第一次触发规则信息
RockIvaBaObjectInfo	单个目标检测基本信息
RockIvaBaResult	检测结果全部信息

3.3.2.3.1 RockIvaBaWireRule

【功能】

越界规则

【声明】

```
typedef struct {  
    uint8_t ruleEnable;  
    uint32_t ruleID;  
    RockIvaLine line;  
    RockIvaBaTripEvent event;  
    RockIvaSize minObjSize[ROCKIVA_OBJECT_TYPE_MAX];  
    RockIvaSize maxObjSize[ROCKIVA_OBJECT_TYPE_MAX];  
    uint8_t scores[ROCKIVA_OBJECT_TYPE_MAX];  
    uint32_t objType;  
    uint8_t rulePriority;  
    uint8_t sense;  
    uint8_t triggerMode;  
} RockIvaBaWireRule;
```

【成员】

成员	描述
ruleEnable	规则是否启用，1->启用，0->不启用
ruleID	规则 ID，有效范围[0， 3]
line	越界线配置
event	越界方向
minObjSize	万百分比表示 最小目标：0 机动车 1 非机动车 2 行人
maxObjSize	万百分比表示 最大目标：0 机动车 1 非机动车 2 行人
scores	各类别过滤分数阈值，0 为内部自动
objType	配置置触发目标类型：如
rulePriority	规则优先级： 0 高， 1 中， 2 低
sense	灵敏度,1~100
triggerMode	触发模式： 0： 目标仅触发一次；1： 目标每次越界都触发

3.3.2.3.2 RockIvaBaAreaRule**【功能】**

区域规则

【声明】

```
typedef struct {  
    uint8_t ruleEnable;  
    uint32_t ruleID;  
    RockIvaArea area;  
    RockIvaSize minObjSize[ROCKIVA_OBJECT_TYPE_MAX];  
    RockIvaSize maxObjSize[ROCKIVA_OBJECT_TYPE_MAX];  
    uint8_t scores[ROCKIVA_OBJECT_TYPE_MAX];  
    uint32_t objType;  
    uint32_t alertTime;  
    uint8_t sense;  
    uint8_t checkEnter;  
} RockIvaBaAreaRule;
```

【成员】

成员	描述
ruleEnable	规则是否启用，1->启用，0->不启用
ruleID	规则 ID，有效范围[1, 256]
area	区域配置
minObjSize	万分比表示 最小目标：0 机动车 1 非机动车 2 行人
maxObjSize	万分比表示 最大目标：0 机动车 1 非机动车 2 行人
scores	各类别过滤分数阈值，0 为内部自动
objType	配置触发目标类型
alertTime	区域入侵规则告警时间设置
sense	灵敏度, 1~100
checkEnter	区域入侵规则是否需要检查目标有进入 [0: 不启用, 1: 启用]

3.3.2.3.3 RockIvaBaTaskRule**【功能】**

周界规则配置

【声明】

```
typedef struct {  
    RockIvaBaWireRule tripWireRule[ROCKIVA_BA_MAX_RULE_NUM];  
    RockIvaBaAreaRule areaInRule[ROCKIVA_BA_MAX_RULE_NUM];  
    RockIvaBaAreaRule areaOutRule[ROCKIVA_BA_MAX_RULE_NUM];  
    RockIvaBaAreaRule areaInBreakRule[ROCKIVA_BA_MAX_RULE_NUM];  
} RockIvaBaTaskRule;
```

【成员】

成员	描述
tripWireRule	越界事件
areaInRule	进入区域
areaOutRule	离开区域
areaInBreakRule	区域入侵

3.3.2.3.4 RockIvaBaAiConfig

【功能】

算法配置

【声明】

```
typedef struct {  
    uint8_t filterPersonMode;  
    uint8_t detectResultMode;  
} RockIvaBaAiConfig;
```

【成员】

成员	描述
filterPersonMode	过滤非机动车/机动车驾驶员人形结果 0：不过滤；1：过滤
detectResultMode	上报目标检测结果模式 0：不上报没有触发规则的检测目标；1：上报没有触发规则的检测目标

3.3.2.3.5 RockIvaBaTaskParams

【功能】

周界初始化参数配置

【声明】

```
typedef struct {  
    RockIvaBaTaskRule baRules;  
    RockIvaBaAiConfig aiConfig;  
} RockIvaBaTaskParams;
```

【成员】

成员	描述
baRules	周界规则参数配置初始化
aiConfig	算法配置

3.3.2.3.6 RockIvaBaTrigger

【功能】

目标第一次触发规则信息，可以用于抓拍

【声明】

```
typedef struct {  
    int32_t ruleID;  
    RockIvaBaRuleTriggerType triggerType;  
} RockIvaBaTrigger;
```

【成员】

成员	描述
ruleID	触发规则 ID
triggerType	触发规则类型

3.3.2.3.7 RockIvaBaObjectInfo

【功能】

触发周界规则的单个目标信息

【声明】

```
typedef struct {  
    RockIvaObjectInfo objInfo;  
    uint32_t triggerRules;  
    uint32_t triggerRulesNum;  
    uint32_t triggerRulesID[ROCKIVA_BA_MAX_RULE_NUM];  
    RockIvaBaTrigger firstTrigger;  
} RockIvaBaObjectInfo;
```

【成员】

成员	描述
objInfo	目标检测结果信息
triggerRules	目标触发的规则
triggerRulesNum	目标触发规则数量
triggerRulesID	目标触发的所有规则 ID
firstTrigger	目标第一次触发的规则

3.3.2.3.8 RockIvaBaResult

【功能】

检测结果全部信息

【声明】

```
typedef struct {  
    uint32_t frameId;  
    RockIvaImage frame;  
    uint32_t channelId;  
    uint32_t objNum;  
    RockIvaObjectInfo triggerObjects [ROCKIVA_MAX_OBJ_NUM];  
} RockIvaBaResult;
```

【成员】

成员	描述
frameId	输入图像帧 ID
frame	对应的输入图像帧
channelId	通道号
objNum	目标个数
triggerObjects	触发周界规则的目标